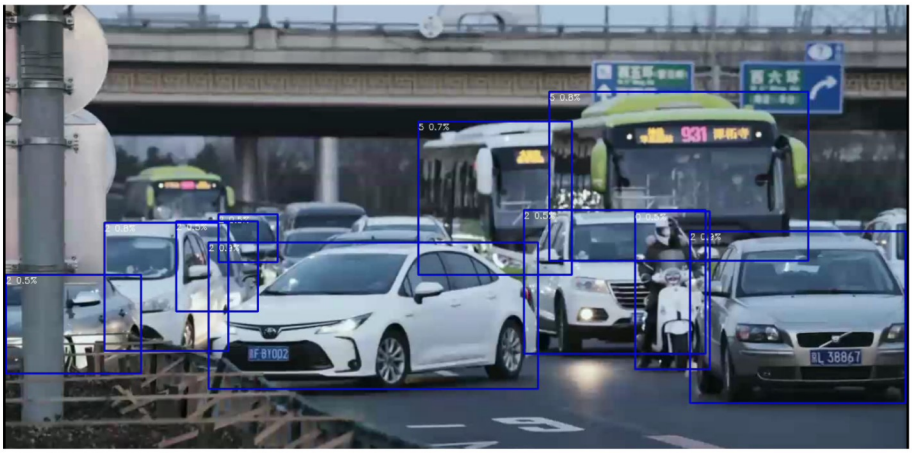


简单上手rk3588系列：（二）YOLOv8n多线程173帧性能复现

pogg
白天当保安拦外卖，晚上送外卖给保安拦

15 人赞同了该文章 ›

【前言】过去一年很难挤出完整的时间，今年适当找点时间，简单更新下以前学习的博客，一是从24年开始，觉得端侧国产芯片没比国外差多少，二是隐隐觉得国内的智能爆发期快到了，长期看好。

读者会不定时更新瑞芯微+/爱芯元智+/地平线+以及其他国产芯片的学习内容，不当之处请包涵~

一、背景

依稀记得22年毕设基于rv1126+做了一套电梯内违禁物品识别的系统，后来又入手了一块3588，同在10TOPS级别以下，性价比颇高。

在学习rk期间，私以为找到了很不错的两个工程repo：

https://github.com/leafqycc/rknn-cpp-Multithreading
github.com/leafqycc/rknn-cpp-Multithreading

https://github.com/455670288/rknn-yolov8s-multi-thread-inference
github.com/455670288/rknn-yolov8s-multi-thread-inferen

尤其是第一个，很值得学习（特别是线程池+和指针调度+）。

二、踩坑

当年不知天高地厚的笔者还试图妄想作一番优化，好刷一下为数不多的虚荣心，但踩了坑后发现，确实也没

关于作者
pogg
白天当保安拦外卖，晚上送...
回答 15 文章 31 关注者 5,077
关注他 发私信
大家都在搜 换一换
哪吒汽车创始人已成老赖 466 万 热
DeepSeek V4 预览版上线... 363 万 热
男子性情大变确诊神经梅毒... 306 万 热
AC 米兰国米球员被曝集体... 288 万 热
GPT-Image-2发布 271 万
OpenAI发布GPT-5.5 255 万
美伊均表示已准备好重启作... 241 万
儿童近视「神药」爆火 年... 237 万
常年穿袜子睡觉睡出「千层... 233 万
华谊兄弟被正式申请破产 225 万

登录即可查看 超5亿 专业优质内容
超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。
立即登录/注册

▲ 赞同 15 ▼ 添加评论 45 喜欢 分享 申请转载 ···

1. 并行加速指针配 C

在工作中学习了汇编加速，想着给工程写一套完全只有asm指令汇编的前处理代码，但因为线程池调度的问题没搞明白，导致前处理流出的数据都是前一帧穿插了另几帧的数据，到现在也没搞明白。

2. 并行不等于加速，搬数据的损耗可能大于计算强度

比如在后处理的源码中，如若强行在网格解码中刷写成并行的方式，耗时不降反增。为啥？细心的网友应该发现了，在并行模块的前一部分，存在着逻辑比较，实际上对于刷写的那块并行，计算强度小，处理数据小，更要命的是每间隔一个grid_len长度（比如6400个元素）取数，没有现成的指令，只能硬搬。这就导致了资源有很大一部分浪费在了搬数据上。

```
for (uint8_t i = 0; i < grid_h; i++)
        {
            for (uint8_t j = 0; j < grid_w; j++)
            {
                int offset = i * grid_w + j;
                int8_t max_class_id = -1;

                if (score_sum_tensor != nullptr)
                {
                    if (score_sum_tensor[offset] < score_sum_thres_i8)
                    {
                        continue;
                    }
                }

                int8_t max_score = -score_zp;
                for (int8_t c = 0; c < OBJ_CLASS_NUM; c++)
                {
                    if ((score_tensor[offset] > score_thres_i8) && (score_tensor[
                    {
                        max_score = score_tensor[offset];
                        max_class_id = c;
                    }
                    offset += grid_len;
                }

                // compute box
                if (max_score > score_thres_i8)
                {
                    offset = i * grid_w + j;
                    float box[4];
                    float before_dfl[dfl_len * 4];

                    uint8_t simd_width = 8;
                    float16x8x2_t _before_dfl_2v_;
                    for (size_t k = 0; k < 4; k++)
                    {
                        int8_t arr0[simd_width] = {
                            box_tensor[offset + (2 * simd_width * k) * gri
                            box_tensor[offset + (2 * simd_width * k + 1) *
                            box_tensor[offset + (2 * simd_width * k + 2) *
```

登录即可查看 超5亿 专业优质内容
超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。
立即登录/注册

```
                    };
                    _before_dfl_2v_.val[0] = vsubq_f16(vcvtq_f16_s16(vmovl_s8
                                                        vcvtq_f16_s16(vdupq_n_
                    _before_dfl_2v_.val[0] = vmulq_f16(_before_dfl_2v_.val[0]
                                                        vdupq_n_f16(float16_t(

                    int8_t arr1[simd_width] = {
                        box_tensor[offset + (2 * simd_width * k + 8) * grid_l
                        box_tensor[offset + (2 * simd_width * k + 9) * grid_l
                        box_tensor[offset + (2 * simd_width * k + 10) * grid_
                        box_tensor[offset + (2 * simd_width * k + 11) * grid_
                        box_tensor[offset + (2 * simd_width * k + 12) * grid_
                        box_tensor[offset + (2 * simd_width * k + 13) * grid_
                        box_tensor[offset + (2 * simd_width * k + 14) * grid_
                        box_tensor[offset + (2 * simd_width * k + 15) * grid_
                    };
                    _before_dfl_2v_.val[1] = vsubq_f16(vcvtq_f16_s16(vmovl_s8
                                                        vcvtq_f16_s16(vdupq_n_
                    _before_dfl_2v_.val[1] = vmulq_f16(_before_dfl_2v_.val[1]
                                                        vdupq_n_f16(box_scale)

                    box[i] = compute_dfl_ps(_before_dfl_2v_, dfl_len);
                }
                for (uint8_t k = 0; k < dfl_len * 4; k++)
                {
                    before_dfl[k] = deqnt_affine_to_f32(box_tensor[offset], b
                    offset += grid_len;
                }
                compute_dfl(before_dfl, dfl_len, box);

                float x1, y1, x2, y2, w, h;
                x1 = (-box[0] + j + 0.5) * stride;
                y1 = (-box[1] + i + 0.5) * stride;
                x2 = (box[2] + j + 0.5) * stride;
                y2 = (box[3] + i + 0.5) * stride;
                w = x2 - x1;
                h = y2 - y1;
                boxes.push_back(x1);
                boxes.push_back(y1);
                boxes.push_back(w);
                boxes.push_back(h);

                objProbs.push_back(deqnt_affine_to_f32(max_score, score_zp, s
                classId.push_back(max_class_id);
                validCount++;
            }
        }
    }
```

再比如，求解dfl，使用64个元素作为一组周期因子送入函数中，理论上最大并行可以
理8个元素。但，实际是否可行？答案是不行，我们可以单独拆解出softmax算子中的
部分，假设塞入类型int8x8_t的tensor，数值为[-13，24，16，7，16，-8，21，-1
过指数运算后，分别为[0.000002241838109481， 26,893,894,496.16517105，
8,976,405.528521011864，1,101.49448801189888，8,976,405.528521011864，
0.00033377112897

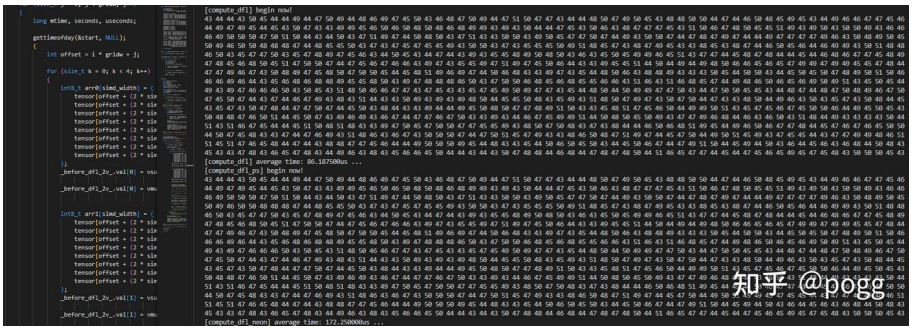
登录即可查看 超5亿 专业优质内容
超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。
立即登录/注册

```
float compute_dfl_ps(float16x8x2_t _before_dfl_2v_, uint8_t dfl_len)
{
  float acc_sum;
  float exp_sum;

  // dfl convulation weights
  const float indices[16] = {0.0f, 1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f,
                             8.0f, 9.0f, 10.0f, 11.0f, 12.0f, 13.0f, 14.0f, 15.0f

  // data load
  const float32x4_t _index_0_ = vld1q_f32(indices);
  const float32x4_t _index_1_ = vld1q_f32(indices + 4);
  const float32x4_t _index_2_ = vld1q_f32(indices + 8);
  const float32x4_t _index_3_ = vld1q_f32(indices + 12);

  // data transform
  float32x4_t _vec_0_ = vcvt_f32_f16(vget_low_f16(_before_dfl_2v_.val[0]));
  float32x4_t _vec_1_ = vcvt_f32_f16(vget_high_f16(_before_dfl_2v_.val[0]));
  float32x4_t _vec_2_ = vcvt_f32_f16(vget_low_f16(_before_dfl_2v_.val[1]));
  float32x4_t _vec_3_ = vcvt_f32_f16(vget_high_f16(_before_dfl_2v_.val[1]));

  // exp sum
  float32x4_t _exp_0_ = exp_ps(_vec_0_);
  float32x4_t _exp_1_ = exp_ps(_vec_1_);
  float32x4_t _exp_2_ = exp_ps(_vec_2_);
  float32x4_t _exp_3_ = exp_ps(_vec_3_);

  exp_sum = vaddvq_f32(_exp_0_) + vaddvq_f32(_exp_1_) + vaddvq_f32(_exp_2_) + vad
  float32x4_t exp_sum_vec = vdupq_n_f32(exp_sum);

  // acc sum
  asm volatile(
      "fdiv v0.4s, %[_exp_0_].4s, %[exp_sum].4s\n"
      "fdiv v1.4s, %[_exp_1_].4s, %[exp_sum].4s\n"
      "fdiv v2.4s, %[_exp_2_].4s, %[exp_sum].4s\n"
      "fdiv v3.4s, %[_exp_3_].4s, %[exp_sum].4s\n"

      "fmul v0.4s, v0.4s, %[_index_0_].4s\n"
      "fmul v1.4s, v1.4s, %[_index_1_].4s\n"
      "fmul v2.4s, v2.4s, %[_index_2_].4s\n"
      "fmul v3.4s, v3.4s, %[_index_3_].4s\n"

      "fadd v4.4s, v0.4s, v1.4s\n"
      "fadd v5.4s, v2.4s, v3.4s\n"
      "fadd v6.4s, v4.4s, v5.4s\n"
      "faddp %[acc].4s, v6.4s, v6.4s\n"

      : [acc] "=w"(acc_sum)
      : [_exp_0_] "w"(_exp_0_),
        [_exp_1_] "w"(_exp_1_),
        [_exp_2_] "w"(_exp_2_),
        [_exp_3_] "w"(_exp_3_),
        [exp_sum] "w"(exp_sum_vec),
        [_index_0_] "w"(_index_0_),
        [_index_
        [_index_
```

登录即可查看 超5亿 专业优质内容
超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。
立即登录/注册

```
    return acc_sum;
  }
```

实际上在整个模块中走并行反而成了负优化，这也是我们需要在实际生产中留意的，不是并行就一定能加速，要多方考虑。



源码/汇编代码结果一致，耗时多了一倍

3. 多核异构/多线程不等于真的流水并行

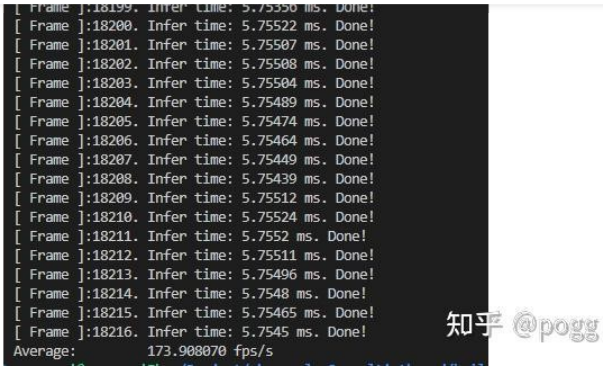
相信很多芯片在宣发上说的并行是无限接近并行，离真并行还有一段距离，即并行跑多个模型（同结构同大小但训练数据不同），结果输出在同一时间戳，相信在不久将来，随着多核调度机制更加完善，这一块应该会很快突破。

三、复现结果

这篇博客基本是跟着大佬的脚步学习，两个repo在工程上也给了笔者不少启发，下附推理结果和延迟。

```
[ Frame ]:18181. Infer time: 5.75535 ms. Done!
[ Frame ]:18182. Infer time: 5.7552 ms. Done!
[ Frame ]:18183. Infer time: 5.7551 ms. Done!
[ Frame ]:18184. Infer time: 5.755 ms. Done!
[ Frame ]:18185. Infer time: 5.75496 ms. Done!
[ Frame ]:18186. Infer time: 5.75509 ms. Done!
[ Frame ]:18187. Infer time: 5.7551 ms. Done!
[ Frame ]:18188. Infer time: 5.75495 ms. Done!
[ Frame ]:18189. Infer time: 5.7548 ms. Done!
[ Frame ]:18190. Infer time: 5.75465 ms. Done!
```

登录即可查看 超5亿 专业优质内容
超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。
立即登录/注册



四、思考

这几年大模型的风吹满地，很多cver爆发了前所未有的焦虑，但我认为端侧小模型在这几年持续发力还是非常重要，毕竟很多业务场景并不需要那么高的参数量，也没有那么高的落地经费，成本控制和毛利依旧是贯穿始终的主旋律（Maybe ┐－_－┐）

工程地址：

github.com/ppogg/YOLOv8...

编辑于 2025-05-08 11:17 · 广东

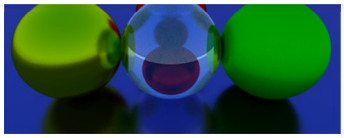
多线程

未登录用户

还没有评论，发表第一个评论吧

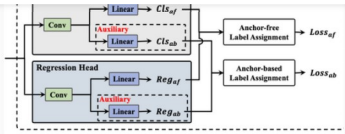
登录即可查看 超5亿 专业优质内容
超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。
立即登录/注册

推荐阅读



用多线程为光线追踪提速

Asixa

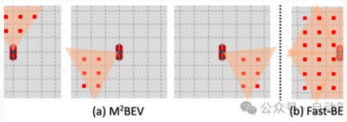


简单上手rk3588系列：（三）YOLOv6n多线程250帧性能...

pogg

【GPU】Turing架构（一）：从GPU-Driven到Mesh...

小小木同子 发表于GPU架构...



FastBEV作者亲自解读: 中低算力平台友好的环视特征融合方案

一介书生 发表于自动驾驶量...

登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。

立即登录/注册